Universidad de Cundinamarca

CADI: Deep Learning — Conceptos

Redes Neuronales V

Reconocimiento facial con redes neuronales siamesas y Triplet Loss
con PyTorch y TensorFlow + Keras

Estudiante: Ruby Dayana Cárdenas Gómez

Docente: Nidia Stella García Roa

Fecha: Noviembre de 2026

Repositorio: https://github.com/RubyDayana/redes-neuronales-v

Contenido

1. Introducción y objetivos
2. Marco conceptual
3. Dataset facial y consentimiento
4. Preprocesamiento de las imágenes
5. Arquitectura de la red siamesa
6. Triplet Loss y entrenamiento
7. Resultados de las pruebas de reconocimiento facial
8. Comparación entre PyTorch y Keras
9. Evidencias de ejecución
10. Conclusiones y limitaciones
11. Repositorio en GitHub
12. Referencias

1. Introducción y objetivos

El reconocimiento facial con aprendizaje profundo no se plantea como una clasificación de personas con una neurona de salida por persona, porque ese enfoque obliga a reentrenar la red cada vez que aparece alguien nuevo. Las **redes neuronales siamesas** resuelven el problema de otra manera: aprenden a convertir cada rostro en un vector (*embedding*) tal que las fotos de la misma persona quedan cerca y las de personas distintas quedan lejos. Reconocer es entonces comparar distancias.

En este taller implementé una red siamesa funcional para reconocimiento facial en **PyTorch** y en **TensorFlow + Keras**, entrenada con **Triplet Loss**, sobre un dataset propio de 38 rostros de tres personas que autorizaron el uso de sus fotografías para fines académicos. Los objetivos fueron:

- Construir y preprocesar un dataset de imágenes faciales (detección, alineación y recorte).
- Implementar la arquitectura siamesa y la Triplet Loss, y comprobar que la pérdida está bien calculada.
- Probar el reconocimiento: **verificación** (¿son la misma persona?) e **identificación** (¿quién es?).
- Evaluar los escenarios de **one-shot**, **few-shot** y **zero-shot learning**.
- Comparar las dos implementaciones y explicar los resultados, incluidos sus límites.

El trabajo está organizado en tres notebooks de Google Colab, que están en el repositorio de GitHub (sección 11):

Tabla 1. Notebooks desarrollados

Notebook	Contenido
01 DatasetFacialPreprocesamiento.ipynb	Auditoría de las fotos, detección con MTCNN, alineación, recorte, control de calidad y dataset final.
02 SiamesaPyTorchTripletLoss.ipynb	Red siamesa en PyTorch, Triplet Loss, pruebas de verificación e identificación, 10 repartos, one-shot, few-shot y zero-shot.
03 SiamesaKerasTripletLoss.ipynb	La misma red en TensorFlow + Keras con modelo siamés de tres ramas, pruebas y comparación con PyTorch.

2. Marco conceptual

2.1. Redes neuronales siamesas y comparación de embeddings

Una red siamesa es una misma red *f* (con los mismos pesos, de ahí el nombre) que se aplica a dos o más imágenes. Cada imagen se transforma en un embedding *f(x)* y la decisión se toma comparando esos vectores: si $\|f(x_1) - f(x_2)\|$ es menor que un umbral, se dice que son la misma persona; para identificar, se compara con los embeddings de las personas registradas y se elige la más cercana (Koch et al., 2015). Como la red aprende una **medida de parecido** y no una lista de personas, puede funcionar con personas que no vio durante el entrenamiento, que es lo que hace útil esta arquitectura en reconocimiento facial (Schroff et al., 2015).

La **comparación de embeddings** es la pieza central: en este trabajo los embeddings tienen 64 números y se normalizan para que tengan longitud 1, de modo que la distancia al cuadrado entre dos de ellos está entre 0 y 4 y no se puede reducir artificialmente achicando los vectores.

2.2. El papel del preprocesamiento

Una red siamesa debe comparar **caras**, no fondos, ropa ni inclinaciones de la cámara. Por eso las fotos se detectan, se alinean y se recortan antes de entrenar. Además, se normalizan los píxeles y se aplica aumento de datos durante el entrenamiento, porque con pocas fotos la red tiende a memorizarlas.

2.3. Triplet Loss

La Triplet Loss trabaja con **tripletas**: un ancla *a*, una imagen positiva *p* (misma persona) y una negativa *n* (otra persona). Exige que el ancla esté más cerca de la positiva que de la negativa por lo menos un margen α:

$$L(a, p, n) = \max(0 , \ \|f(a) - f(p)\|^2 - \|f(a) - f(n)\|^2 + \alpha)$$

Si la tripleta ya cumple la condición, la pérdida es 0 y no aporta; si no, la red se corrige. Según su dificultad, las tripletas son **fáciles** (la negativa ya está más lejos que la positiva más el margen; pérdida 0), **semi-duras** (la negativa está más lejos que la positiva, pero dentro del margen) o **duras** (la negativa está más cerca que la positiva). Como la mayoría de las tripletas posibles son fáciles y no enseñan nada, se eligen dentro del lote (*mining*): **batch-hard** usa para cada ancla su positiva más lejana y su negativa más cercana (Hermans et al., 2017), y **batch-all** usa todas las tripletas válidas del lote y promedia las que siguen activas. En este taller se probaron las dos.

2.4. Verificación, identificación y one-shot, few-shot y zero-shot

La **verificación** (1:1) decide si dos fotos son de la misma persona; se mide con el AUC (área bajo la curva ROC; 1 es perfecto y 0.5 es azar), el EER (tasa de error donde los falsos positivos y los falsos negativos se igualan) y la exactitud con un umbral. La **identificación** (1:N) decide de quién es una foto entre las personas registradas.

Tabla 2. Diferencias entre one-shot, few-shot y zero-shot

Concepto	Definición	Cómo se probó aquí
One-shot	Reconocer a una persona a partir de una sola foto de referencia.	Cada persona se registra con 1 foto de entrenamiento elegida al azar (30 sorteos por reparto) y se identifican las fotos de prueba.
Few-shot	Lo mismo, con pocas fotos de referencia.	Registro con 2, 3 y 5 fotos por persona, sin reentrenar la red.
Zero-shot	Reconocer a una persona que la red nunca vio durante el entrenamiento, sin ajustar pesos.	Se deja a una persona fuera del entrenamiento y se compara con sus embeddings (verificación e identificación con una foto).

2.5. Calidad y organización de los datasets faciales

La calidad del dataset pesa más que la del modelo. Una sola foto mal asignada contamina toda una identidad: en este trabajo, por ejemplo, una captura de pantalla cuyo rostro principal era de otra persona habría quedado dentro de una carpeta ajena si no se hubieran revisado los recortes. También importan la resolución del rostro, la nitidez, la iluminación, la variedad de poses y el equilibrio entre personas. Por eso el dataset se organizó con una carpeta por persona y un archivo de metadatos con la confianza de detección, el tamaño, la nitidez y el brillo de cada rostro.

2.6. Aplicaciones en visión computacional

Las redes siamesas se usan en verificación de identidad (desbloqueo facial, control de acceso), reidentificación de personas entre cámaras, comparación de firmas y de huellas, búsqueda de imágenes similares y reconocimiento de objetos con muy pocos ejemplos, siempre que el problema se pueda plantear como "decidir si dos cosas son iguales".

3. Dataset facial y consentimiento

El dataset se construyó con **42 fotografías personales de tres personas**: la autora del trabajo y dos personas cercanas que, según declara la estudiante, autorizaron el uso de sus imágenes únicamente para fines académicos de esta actividad. Por privacidad las carpetas llevan un apodo (ayis, gaby, yeyo), las fotos originales no se publican en el repositorio (además traen metadatos de ubicación) y los recortes se usan solo para este taller.

Las fotos son de celulares distintos, tomadas entre 2016 y 2026; su tamaño va de 0.03 a 12 megapíxeles, porque varias se recibieron por WhatsApp. Esa variedad natural de edad, peinado, iluminación y calidad es útil para probar el modelo, pero lo hace más difícil con tan pocos ejemplos.

Tabla 3. Fotos originales y rostros del dataset final

Persona	Fotos originales	Rostros en el dataset	Lado mediano del rostro (px)	Brillo medio (0–255)
ayis	17	17	424.8	80.6
gaby	9	7	253.1	89.3
yeyo	16	14	233.3	111.2
Total	42	38		

Se descartaron cuatro fotos: dos fotos de grupo de gaby en las que no se puede asegurar cuál de las tres caras es ella, una foto de yeyo con un rostro de solo 34 píxeles, y una captura de pantalla cuyo rostro principal era de otra persona. El desbalance entre personas (17, 7 y 14 rostros) es una limitación que se tiene en cuenta al evaluar, usando la exactitud balanceada.

4. Preprocesamiento de las imágenes

El preprocesamiento se hizo en el notebook 1 con los siguientes pasos:

1. **Corrección de orientación** (EXIF) y reducción de las fotos muy grandes a un máximo de 1280 píxeles.
2. **Detección con MTCNN** (Zhang et al., 2016), una red en cascada de tres etapas que devuelve el recuadro de la cara, una confianza y cinco puntos de referencia (ojos, nariz y comisuras de la boca). Si hay varias caras se toma la de mayor área. Se exige confianza ≥ 0.90 y un lado mínimo de 40 píxeles.
3. **Alineación:** con la posición de los ojos se calcula el ángulo de la línea que los une y se rota la imagen para dejarla horizontal.
4. **Recorte** cuadrado con 30 % de margen alrededor del recuadro y redimensionado a 112 × 112 píxeles (64 × 64 para entrenar).
5. **Normalización** de los píxeles a (x/255 − 0.5)/0.25 con constantes fijas, sin usar datos de prueba, y **aumento de datos** solo en entrenamiento: giro horizontal, recorte aleatorio, rotación de ±10°, cambios de brillo, contraste y color, y escala de grises ocasional.

MTCNN encontró cara en las 42 fotos (confianza mediana 0.9999), pero 7 fotos tenían más de una cara. La línea de los ojos estaba inclinada más de 5° en 26 de las 39 fotos conservadas para el análisis, con una mediana de 15.5° y un máximo de 47.3°, así que la alineación es necesaria (Figura 2).



Figura 1. Detección con MTCNN: recuadro de la cara y cinco puntos de referencia en una foto de cada persona.

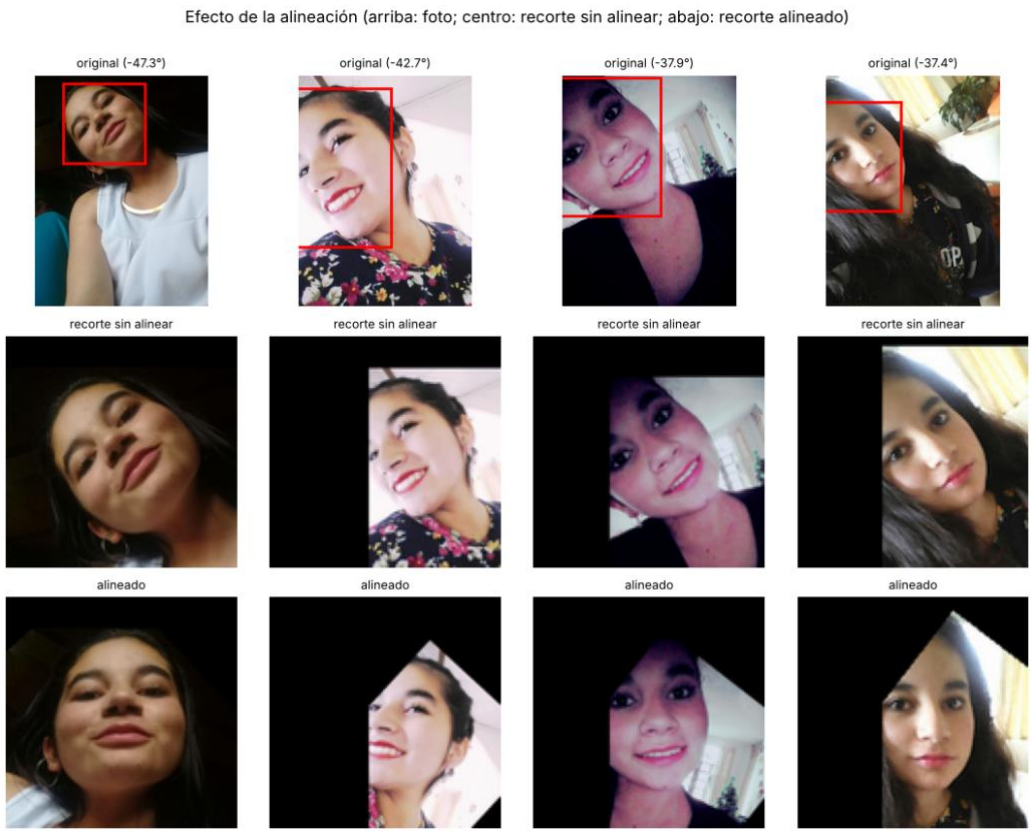


Figura 2. Efecto de la alineación en las cuatro fotos más inclinadas (arriba: foto original; centro: recorte sin alinear; abajo: recorte alineado). Las zonas negras son el relleno que aparece al rotar cerca del borde de la foto.

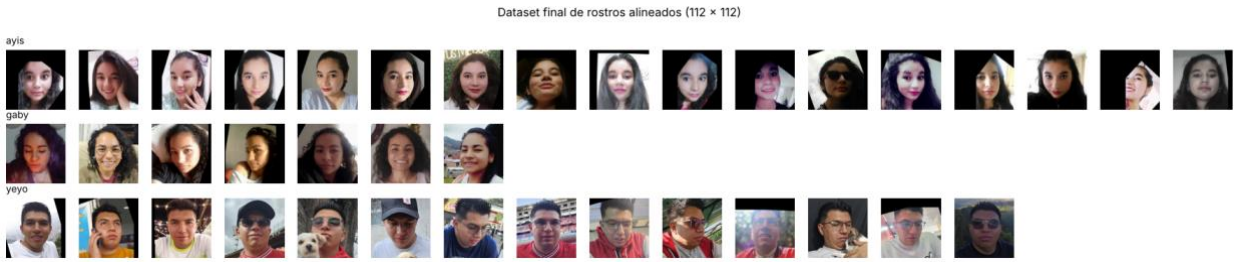


Figura 3. Dataset final: 38 rostros alineados de 112 × 112 píxeles, una fila por persona.

En el control de calidad, 5 de los 38 rostros son pequeños (lado menor de 80 px en la foto original, es decir, ampliados) y ninguno está desenfocado según el umbral usado (nitidez mayor que 100). Las fotos de ayis son en promedio las más oscuras (brillo 81 sobre 255, frente a 89 y 111).

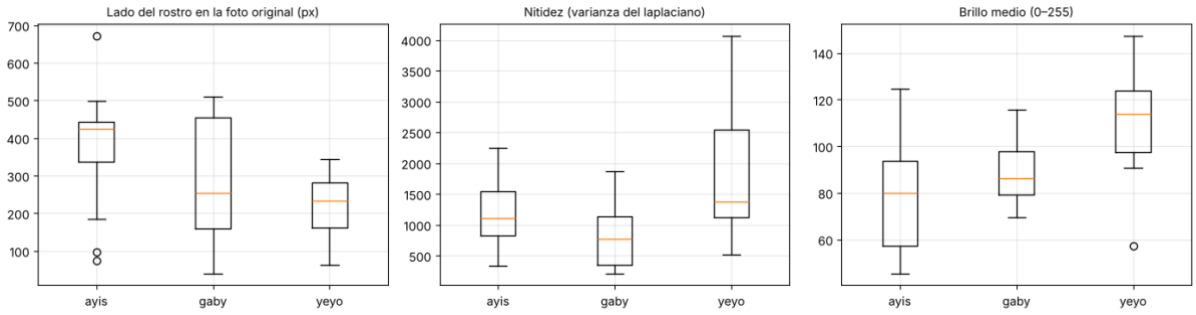


Figura 4. Calidad de los rostros por persona: tamaño del rostro en la foto original, nitidez y brillo.

Código fuente de la alineación y el recorte:

```
def alinear_y_recortar(im, caja, puntos, tam=TAM, margen=MARGEN):
    ojo_i, ojo_d = puntos[0], puntos[1]
    angulo = math.degrees(math.atan2(ojo_d[1] - ojo_i[1], ojo_d[0] - ojo_i[0]))   # inclinación de la línea de los ojos
    cx, cy = (caja[0] + caja[2]) / 2, (caja[1] + caja[3]) / 2
    girada = im.rotate(angulo, resample=Image.BICUBIC, center=(cx, cy), fillcolor=(0, 0, 0))
```

```
lado = max(caja[2] - caja[0], caja[3] - caja[1]) * (1 + 2 * margen)
recorte = girada.crop((cx - lado / 2, cy - lado / 2, cx + lado / 2, cy + lado / 2))
return recorte.resize((tam, tam), Image.LANCZOS), angulo
```

5. Arquitectura de la red siamesa

La red base es una CNN pequeña, apropiada para el tamaño del dataset, con unos 250 000 parámetros. Convierte un rostro de 64 × 64 en un embedding de 64 números con norma 1. Cada imagen pasa por **la misma red** con los mismos pesos, que es lo que la hace siamesa.

Tabla 4. Arquitectura de la red base (igual en PyTorch y en Keras)

Parte	Capas	Salida
Entrada	Imagen RGB normalizada	3 × 64 × 64
Bloque 1	Conv 3×3 (32) → BatchNorm → ReLU → MaxPool	32 × 32 × 32
Bloque 2	Conv 3×3 (64) → BatchNorm → ReLU → MaxPool	64 × 16 × 16
Bloque 3	Conv 3×3 (128) → BatchNorm → ReLU → MaxPool	128 × 8 × 8
Bloque 4	Conv 3×3 (128) → BatchNorm → ReLU → MaxPool	128 × 4 × 4
Cabeza	Promedio global → Dropout 0.3 → Lineal (64) → normalización L2	vector de 64 con norma 1

La red tiene 249 440 parámetros entrenables en PyTorch (250 144 en Keras, que cuenta además 704 valores no entrenables de la normalización por lotes). En Keras, además de la red base, se construyó el **modelo siamés de tres ramas** (ancla, positiva y negativa) que aplica la misma instancia de la red base a las tres entradas; se comprobó que los pesos se comparten (la misma imagen da exactamente el mismo embedding en cualquier rama) y que ese modelo no agrega parámetros.

Red siamesa en PyTorch:

```
class RedSiamesa(nn.Module):
    """CNN que convierte un rostro en un embedding de `dim` números con norma 1."""
    def __init__(self, dim=64):
        super().__init__()
        def bloque(i, o):
            return nn.Sequential(nn.Conv2d(i, o, 3, padding=1, bias=False), nn.BatchNorm2d(o), nn.ReLU(inplace=True), nn.MaxPool2d(2))
        self.cuerpo = nn.Sequential(bloque(3, 32), bloque(32, 64), bloque(64, 128), bloque(128, 128))
        self.cabeza = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(0.3), nn.Linear(128, dim))

    def forward(self, x):
        return F.normalize(self.cabeza(self.cuerpo(x)), dim=1)

modelo_demo = RedSiamesa()
print(f"Parámetros entrenables: {sum(p.numel() for p in modelo_demo.parameters() if p.requires_grad):,}")
salida = modelo_demo.eval()(torch.randn(5, 3, TAM, TAM))
print("Salida para 5 imágenes:", tuple(salida.shape), "| normas:", salida.norm(dim=1).detach().numpy().round(3))

Parámetros entrenables: 249,440
Salida para 5 imágenes: (5, 64) | normas: [1. 1. 1. 1. 1.]
```

Red base y modelo siamés de tres ramas en Keras:

```python
def crear_red_base(dim=64):
    entrada = keras.Input((TAM, TAM, 3), name="rostro")
    x = entrada
    for filtros in (32, 64, 128, 128):
        x = layers.Conv2D(filtros, 3, padding="same", use_bias=False)(x)
        x = layers.BatchNormalization()(x)
        x = layers.ReLU()(x)
        x = layers.MaxPooling2D()(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(dim)(x)
    salida = layers.UnitNormalization(name="embedding_L2")(x)          # norma 1
    return keras.Model(entrada, salida, name="red_base")

def crear_modelo_siames(red_base):
    """Tres entradas -> la MISMA red base -> tres embeddings (pesos compartidos)."""
    a, p, n = (keras.Input((TAM, TAM, 3), name=nombre) for nombre in ("ancla", "positiva", "negativa"))
    salida = layers.Concatenate(name="embeddings")([red_base(a), red_base(p), red_base(n)])
    return keras.Model([a, p, n], salida, name="siames_tres_ramas")

red_base = crear_red_base()
siames = crear_modelo_siames(red_base)
print(f"Parámetros de la red base: {red_base.count_params():,}  |  parámetros del modelo de tres ramas: {siames.count_params():,}  (los mismos: los pesos se comparten)")
red_base.summary()
```

```
Parámetros de la red base: 250,144  |  parámetros del modelo de tres ramas: 250,144  (los mismos: los pesos se comparten)
Model: "red_base"
```

6. Triplet Loss y entrenamiento

6.1. Implementación de la pérdida y su verificación

La Triplet Loss se implementó a mano (con la distancia al cuadrado, como en FaceNet) y se comprobó de dos maneras. Primero, contra nn.TripletMarginLoss de PyTorch con 32 tripletas aleatorias: la diferencia absoluta fue de 0. Segundo, con un ejemplo calculado a mano de cada tipo de tripleta (margen 0.3), que reproducen ambos frameworks:

Tabla 5. Ejemplo numérico de los tres tipos de tripleta (margen α = 0.3)

Tripleta	d(a,p)	d(a,n)	Pérdida = máx(0, d(a,p) − d(a,n) + 0.3)
Fácil	0.040	4.000	0.000
Semi-dura	0.040	0.200	0.140
Dura	0.400	0.100	0.600

Triplet Loss en PyTorch:

```python
def triplet_loss(ancla, pos, neg, margen, al_cuadrado=True):
    """Triplet Loss de FaceNet: max(0, d(a,p) - d(a,n) + margen), con d = distancia euclídea (al cuadrado por defecto)."""
    d_ap = (ancla - pos).pow(2).sum(1)
    d_an = (ancla - neg).pow(2).sum(1)
    if not al_cuadrado:
        d_ap, d_an = d_ap.sqrt(), d_an.sqrt()
    return F.relu(d_ap - d_an + margen).mean()
```

Minería de tripletas en el lote, en PyTorch (batch-hard y batch-all):

```python
def perdida_batch_hard(emb, etiquetas, margen):
    """Para cada ancla: positiva más lejana y negativa más cercana dentro del lote. Devuelve (pérdida, fracción activa)."""
    d = torch.cdist(emb, emb).pow(2)
    misma = etiquetas[:, None] == etiquetas[None, :]
    yo = torch.eye(len(etiquetas), dtype=torch.bool)
    d_pos = d.masked_fill(~misma | yo, float("-inf")).max(1).values
    d_neg = d.masked_fill(misma, float("inf")).min(1).values
    perdida = F.relu(d_pos - d_neg + margen)
    return perdida.mean(), (perdida > 0).float().mean()

def perdida_batch_all(emb, etiquetas, margen):
    """Todas las tripletas válidas del lote; la pérdida es el promedio de las que siguen activas."""
    d = torch.cdist(emb, emb).pow(2)
    misma = etiquetas[:, None] == etiquetas[None, :]
    yo = torch.eye(len(etiquetas), dtype=torch.bool)
    valida = (misma & ~yo)[:, :, None] & (~misma)[:, None, :]            # (ancla, positiva, negativa)
    perdida = F.relu(d[:, :, None] - d[:, None, :] + margen) * valida
    activas = (perdida > 0).float().sum()
    return perdida.sum() / activas.clamp(min=1), activas / valida.sum()

PERDIDAS = {"hard": perdida_batch_hard, "all": perdida_batch_all}
```

Minería de tripletas y bucle de entrenamiento en Keras (batch-all, `tf.GradientTape`):

```python
def perdida_batch_all(emb, etiquetas, margen=0.3):
    """Todas las tripletas válidas del lote; la pérdida es el promedio de las que siguen activas."""
    sq = tf.reduce_sum(tf.square(emb), axis=1)
    d = tf.maximum(sq[:, None] - 2 * emb @ tf.transpose(emb) + sq[None, :], 0.0)       # distancias al cuadrado
    misma = etiquetas[:, None] == etiquetas[None, :]
    yo = tf.eye(tf.shape(etiquetas)[0], dtype=tf.bool)
    valida = tf.cast((misma & ~yo)[:, :, None] & (~misma)[:, None, :], tf.float32)
    perdida = tf.nn.relu(d[:, :, None] - d[:, None, :] + margen) * valida
    activas = tf.reduce_sum(tf.cast(perdida > 0, tf.float32))
    return tf.reduce_sum(perdida) / tf.maximum(activas, 1.0), activas / tf.reduce_sum(valida)
```

6.2. Entrenamiento

En cada paso se arma un lote con **6 fotos de cada persona** de entrenamiento, se le aplica aumento de datos, se calculan los embeddings y se aplica la pérdida. Se usa AdamW (tasa de aprendizaje 0.002 con decaimiento coseno, *weight decay* de 1e-4), 300 pasos y margen 0.3. Las fotos de prueba no se usan nunca en el entrenamiento ni para elegir el umbral.

Hallazgo: batch-hard colapsa. Con el mismo reparto de datos, la variante batch-hard no aprendió: la pérdida se quedó en 0.307 (casi el margen), el 100 % de las tripletas siguieron activas y la dispersión de los embeddings fue de 0.006, frente a 0.103 de batch-all (valor esperado sin colapso: cerca de 0.125). Es el colapso clásico: la red da el mismo embedding a todos los rostros, de modo que d(a,p) = d(a,n) = 0 y cada tripleta tiene pérdida igual al margen. Con batch-all, en cambio, la pérdida baja a 0.216 y las tripletas activas pasan de 100 % a 2.6 %. Por eso todo lo demás usa batch-all.

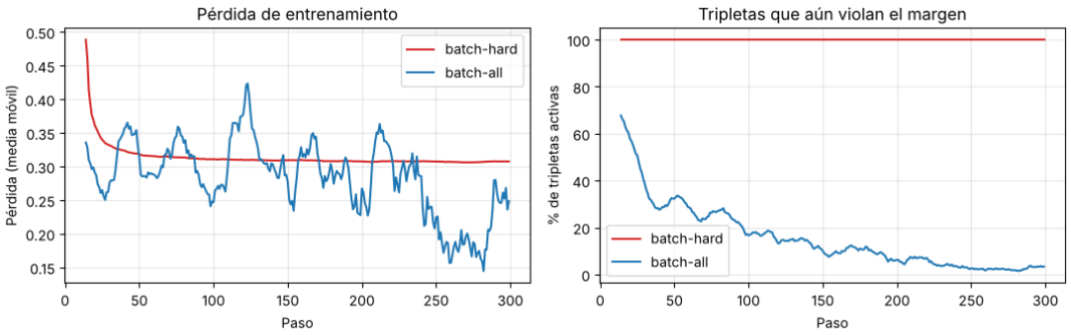


Figura 5. Entrenamiento en PyTorch con batch-hard (rojo) y batch-all (azul): pérdida y porcentaje de tripletas activas.



Figura 6. Aumento de datos aplicado a un rostro de entrenamiento.

Bucle de entrenamiento en PyTorch:

```python
def entrenar(X_tr, y_tr, semilla=0, pasos=300, margen=0.3, dim=64, lr=2e-3, mineria="all", verbose=False):
    """Entrena la red siamesa con Triplet Loss. Devuelve el modelo (en modo evaluación) y el historial."""
    torch.manual_seed(semilla); rng = np.random.RandomState(semilla)
    modelo = RedSiamesa(dim)
    opt = torch.optim.AdamW(modelo.parameters(), lr=lr, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=pasos)
    perdida_fn = PERDIDAS[mineria]
    X_t = torch.from_numpy(X_tr).permute(0, 3, 1, 2)
    historial = {"perdida": [], "activas": []}
    for paso in range(pasos):
        modelo.train()
        idx = muestrear_lote(y_tr, rng)
        lote = (torch.stack([aumento(X_t[i]) for i in idx]).float() / 255 - 0.5) / 0.25
        perdida, activas = perdida_fn(modelo(lote), torch.from_numpy(y_tr[idx]), margen)
        opt.zero_grad(); perdida.backward(); opt.step(); sched.step()
        historial["perdida"].append(perdida.item()); historial["activas"].append(activas.item())
        if verbose and (paso + 1) % 75 == 0:
            print(f"  paso {paso + 1:4d}  pérdida {np.mean(historial['perdida'][-75:]):.4f}  tripletas activas {np.mean(historial['activas'][-75:]) * 100:5.1f} %")
    return modelo.eval(), historial

@torch.no_grad()
```

7. Resultados de las pruebas de reconocimiento facial

Todas las pruebas usan **fotos que la red nunca vio durante el entrenamiento**. El reparto es estratificado (aproximadamente 35 % de las fotos de cada persona para prueba): en el reparto 0 hay 25 fotos de entrenamiento y 13 de prueba (ayis 6, gaby 2, yeyo 5), es decir, 78 pares de prueba, 26 de la misma persona. Para saber si la red realmente aprendió, se compara con dos referencias sin aprendizaje: los **píxeles crudos** (gris 32 × 32, comparación por coseno) y la **misma red sin entrenar** (pesos aleatorios).

7.1. Un reparto de ejemplo (reparto 0, PyTorch)

Tabla 6. Verificación e identificación en el reparto 0, con fotos de prueba (PyTorch)

Modelo	AUC	EER	Exactitud verificación (bal.)	Exactitud identificación (bal.)
Píxeles crudos	0.488	0.529	0.519	0.689
Red sin entrenar	0.496	0.500	0.481	0.522
Siamesa, batch-hard (colapsada)	0.704	0.385	0.606	0.589
Siamesa, batch-all	0.823	0.288	0.750	0.889

La siamesa con batch-all logra un AUC de 0.823: si se escoge un par de la misma persona y uno de personas distintas, el de la misma persona está más cerca en el 82 % de los casos. Identifica **11 de las 13 fotos de prueba**: acierta las 5 de yeyo y las 2 de gaby, y se equivoca en 2 de las 6 de ayis (una la confunde con gaby y otra con yeyo). Las dos referencias dan resultados de azar en verificación (AUC cercano a 0.5). Los píxeles crudos llegan a 0.689 en identificación, algo alto que se explica porque varias fotos de una misma persona comparten fondo, iluminación o ropa.

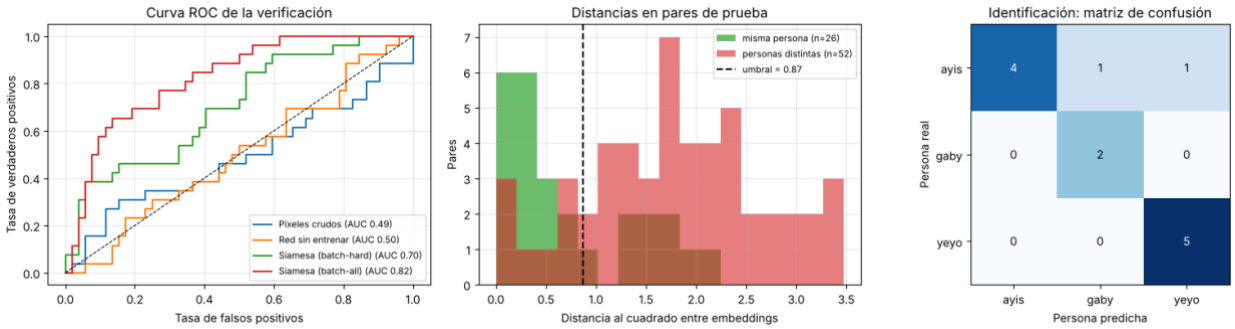


Figura 7. Reparto 0 (PyTorch): curva ROC de la verificación, distribución de distancias en pares de prueba con el umbral elegido en entrenamiento (0.87) y matriz de confusión de la identificación.

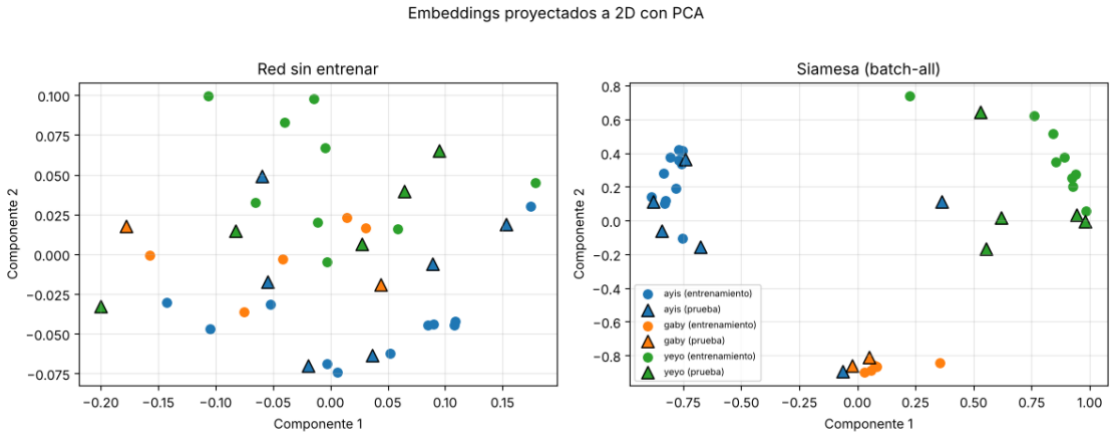


Figura 8. Embeddings proyectados a 2D con PCA: con la red sin entrenar las personas se mezclan; con la red entrenada se agrupan (círculos: entrenamiento; triángulos: prueba).

7.2. Diez repartos distintos

Con 38 fotos, un solo reparto puede salir bien o mal por suerte. Por eso se repitió todo (reparto, entrenamiento y evaluación) con 10 semillas distintas.

Tabla 7. Promedio ± desviación estándar en 10 repartos (PyTorch, fotos de prueba)

Modelo	AUC	EER	Exactitud verificación (bal.)	Exactitud identificación (bal.)
Píxeles crudos	0.615 ± 0.119	0.408 ± 0.098	0.580 ± 0.089	0.629 ± 0.161
Red sin entrenar	0.638 ± 0.089	0.384 ± 0.077	0.562 ± 0.064	0.569 ± 0.197
Siamesa (batch-all)	0.772 ± 0.082	0.299 ± 0.094	0.694 ± 0.087	0.682 ± 0.123

La siamesa supera a los píxeles crudos en AUC en 8 de los 10 repartos y a la red sin entrenar en 9 de 10. En identificación la mejora existe pero es pequeña frente a la dispersión (0.682 frente a 0.629 y 0.569; el azar es 0.333), y el resultado va de 0.54 a 0.89 según qué fotos caigan en prueba. Esa variabilidad se debe al tamaño del dataset: apenas 13 fotos de prueba en cada reparto y solo 2 de gaby.

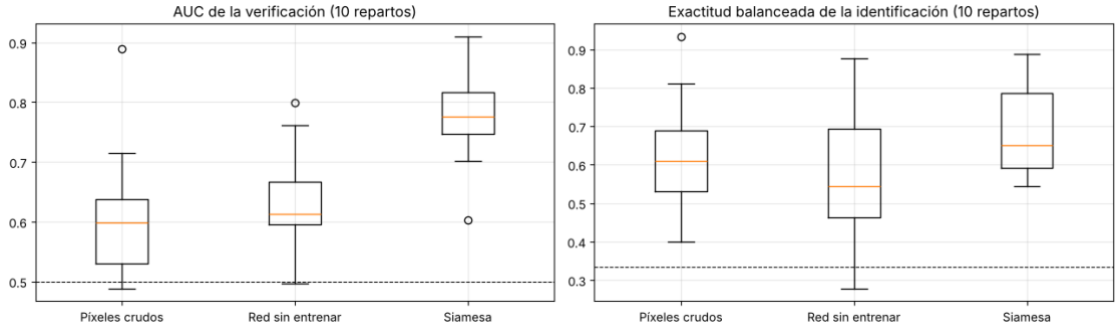


Figura 9. Distribución del AUC y de la exactitud de identificación en 10 repartos para las tres comparaciones (la línea punteada es el azar).

7.3. One-shot y few-shot

Cada persona se registra con 1, 2, 3 o 5 fotos de entrenamiento elegidas al azar y se identifican las fotos de prueba por el prototipo más cercano, sin reentrenar la red.

Tabla 8. Exactitud balanceada de la identificación según las fotos de referencia por persona (PyTorch; azar = 0.333)

Fotos de referencia	Píxeles crudos	Red sin entrenar	Siamesa
1 (one-shot)	0.489 ± 0.055	0.502 ± 0.086	0.704 ± 0.110
2 (few-shot)	0.536 ± 0.062	0.528 ± 0.111	0.702 ± 0.114
3 (few-shot)	0.575 ± 0.066	0.550 ± 0.131	0.697 ± 0.113
5 (few-shot)	0.611 ± 0.092	0.570 ± 0.150	0.690 ± 0.116

La red registra a una persona **con una sola foto** y la identifica correctamente el 70 % de las veces, muy por encima de las referencias sin aprendizaje (49 % y 50 %). Llama la atención que **más fotos de referencia no mejoran** a la siamesa (0.704 → 0.690), mientras que los píxeles sí mejoran (0.489 → 0.611) y acortan la distancia. Una lectura posible es que los errores de la siamesa no se deben a pocas referencias sino a fotos de prueba difíciles (poses extremas, iluminación, lentes) que las referencias adicionales no corrigen.

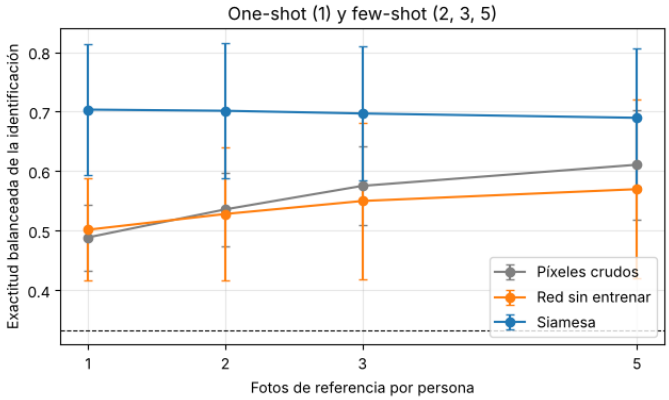


Figura 10. One-shot (1 foto) y few-shot (2, 3 y 5 fotos): identificación con PyTorch, promedio y desviación en 10 repartos.

7.4. Zero-shot: una persona que la red nunca vio

Se deja a una persona completamente fuera del entrenamiento (la red solo ve a las otras dos) y se mide si sus embeddings sirven para distinguirla. En la verificación, los pares (persona no vista, persona no vista) son positivos y los pares (persona no vista, persona conocida) son negativos. Se repite dejando fuera a cada persona y con 5 semillas.

Tabla 9. Zero-shot: AUC de la verificación y fracción de fotos de la persona no vista identificadas con una sola foto de referencia (promedio de 5 semillas)

Persona no vista	AUC píxeles	AUC red sin entrenar	AUC siamesa	Identif. 1 foto, siamesa
ayis	0.641	0.647	0.781	0.411
gaby	0.684	0.603	0.603	0.268
yeyo	0.552	0.582	0.627	0.386
Global	0.626 ± 0.065	0.611 ± 0.095	0.671 ± 0.095	0.355 ± 0.130

El resultado es negativo, y es el esperado: la verificación zero-shot de la siamesa (AUC 0.671) apenas supera a las referencias (0.626 y 0.611), y al identificar con una sola foto reconoce el 35.5 % de las fotos de la persona nueva, frente a 44.9 % de los píxeles crudos y un azar de 33.3 %. **No hay evidencia de que la red generalice a personas nuevas.** La explicación es de datos, no de método: para aprender una noción general de parecido facial hay que ver muchas identidades; aquí cada red aprende con solo dos personas (14 a 20 fotos), lo que alcanza para separarlas entre sí pero no para construir un espacio de embeddings útil para otras caras. Los sistemas reales se entrenan con del orden de millones de identidades (Schroff et al., 2015). La mejor señal es la de ayis (AUC 0.781), la persona con más fotos.

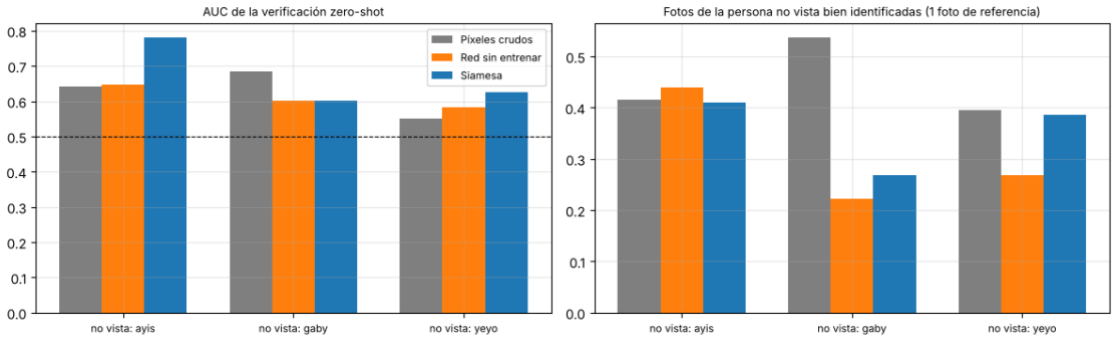


Figura 11. Zero-shot con PyTorch: AUC de la verificación y fracción de la persona no vista identificada con una foto, según la persona que se deja fuera.

8. Comparación entre PyTorch y Keras

La misma arquitectura, la misma Triplet Loss batch-all y los mismos 10 repartos se implementaron en TensorFlow + Keras. Para que los resultados sean idénticos en cada ejecución se activó el modo determinista de TensorFlow y se fijó la semilla en cada entrenamiento.

Tabla 10. PyTorch frente a Keras en 10 repartos (promedio ± desviación estándar)

Medida	PyTorch	Keras	Keras − PyTorch
AUC verificación	0.772 ± 0.082	0.765 ± 0.118	−0.007
EER verificación	0.299 ± 0.094	0.263 ± 0.117	−0.037
Exactitud verificación (bal.)	0.694 ± 0.087	0.702 ± 0.092	+0.008
Exactitud identificación (bal.)	0.682 ± 0.123	0.758 ± 0.115	+0.076
1-shot	0.704 ± 0.110	0.759 ± 0.112	+0.055
2-shot	0.702 ± 0.114	0.762 ± 0.115	+0.061
3-shot	0.697 ± 0.113	0.757 ± 0.120	+0.060
5-shot	0.690 ± 0.116	0.756 ± 0.119	+0.066

En **verificación no hay diferencia apreciable** (AUC 0.772 y 0.765; prueba t pareada p = 0.807). En **identificación Keras queda mejor en promedio** (0.758 frente a 0.682), pero con 10 repartos la diferencia no es estadísticamente concluyente (p = 0.220). Tampoco se puede atribuir al framework: aunque la arquitectura, la pérdida y los repartos son los mismos, la inicialización de pesos (Glorot en Keras, Kaiming en PyTorch), el momento de la normalización por lotes y las capas de aumento de datos no son idénticos. La correlación entre frameworks del AUC por reparto es 0.68: los repartos difíciles lo son en ambos. El entrenamiento tarda unos 18 s por reparto en Keras y unos 24 s en PyTorch, en CPU; con tan pocos datos la diferencia es irrelevante.

Keras: entrenamiento con `tf.GradientTape` :

```
def paso_entrenamiento(red, aumento, optimizador, lote_uint8, etiquetas, margen):
    # el aumento trabaja en escala 0–255; después se normaliza
    lote = normalizar(tf.clip_by_value(aumento(tf.cast(lote_uint8, tf.float32), training=True), 0, 255))
    with tf.GradientTape() as cinta:
        perdida, activas = perdida_batch_all(red(lote, training=True), etiquetas, margen)
    grads = cinta.gradient(perdida, red.trainable_variables)
    optimizador.apply_gradients(zip(grads, red.trainable_variables))
    return perdida, activas

def entrenar(X_tr, y_tr, semilla=0, pasos=300, margen=0.3, dim=64, lr=2e-3, verbose=False):
    keras.utils.set_random_seed(semilla); rng = np.random.RandomState(semilla)
    red = crear_red_base(dim)
    cosine = keras.optimizers.schedules.CosineDecay(lr, pasos)
    opt = keras.optimizers.AdamW(learning_rate=cosine, weight_decay=1e-4)
    opt.build(red.trainable_variables)                    # crea las variables del optimizador ANTES de compilar el paso
    aumento = crear_aumento()
    paso_compilado = tf.function(lambda lote, et: paso_entrenamiento(red, aumento, opt, lote, et, margen))
    hist = {"perdida": [], "activas": []}
    for paso in range(pasos):
        idx = muestrear_lote(y_tr, rng)
        p, a = paso_compilado(tf.constant(X_tr[idx]), tf.constant(y_tr[idx]))
        hist["perdida"].append(float(p)); hist["activas"].append(float(a))
        if verbose and (paso + 1) % 75 == 0:
            print(f"  paso {paso + 1:4d}  pérdida {np.mean(hist['perdida'][-75:]):.4f}  tripletas activas {np.mean(hist['activas'][-75:]) * 100:5.1f} %")
    return red, hist

def incrustar(red, imgs_uint8):
    return red(normalizar(imgs_uint8), training=False).numpy()
```

9. Evidencias de ejecución

Las siguientes imágenes son salidas reales de la ejecución de los notebooks: cada foto de prueba con su persona real, la predicción del modelo y la similitud; y las curvas de entrenamiento y evaluación de Keras. Las salidas completas están guardadas en los notebooks del repositorio.



Figura 12. PyTorch, reparto 0: identificación de las 13 fotos de prueba (verde: acierto; rojo: error). 11 de 13 correctas.

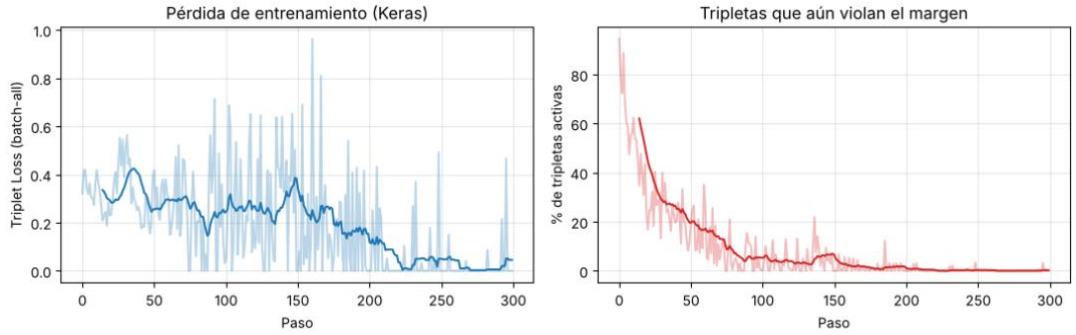


Figura 13. Keras: pérdida de entrenamiento (batch-all) y porcentaje de tripletas activas. Entrenamiento de 300 pasos en 17.7 s.

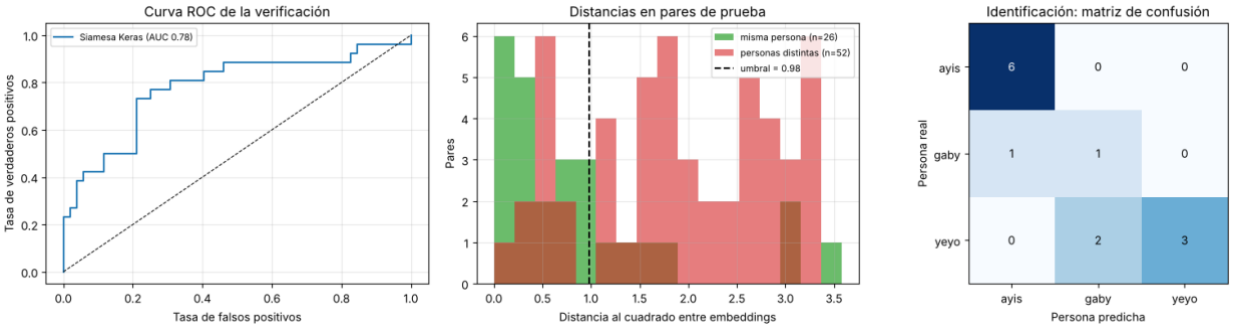


Figura 14. Keras, reparto 0: curva ROC, distancias en pares de prueba y matriz de confusión de la identificación (AUC 0.777; identificación 0.700).



Figura 15. Keras, reparto 0: identificación de las fotos de prueba.

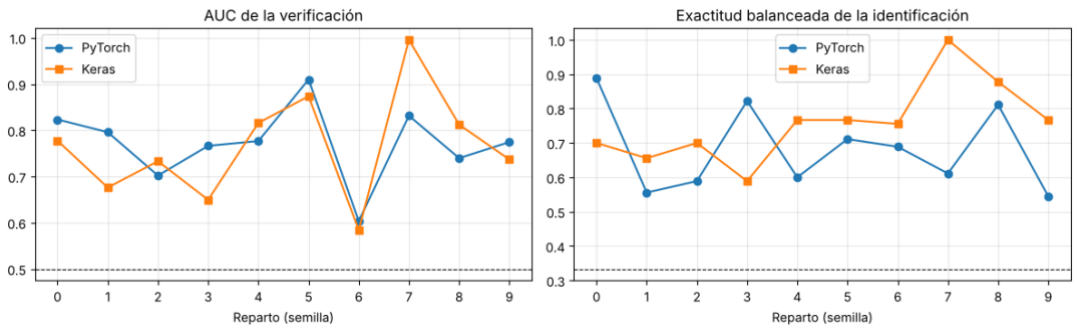


Figura 16. PyTorch frente a Keras: AUC e identificación en cada uno de los 10 repartos.

10. Conclusiones y limitaciones

1. **La red siamesa aprende.** Con Triplet Loss *batch-all* logra un AUC de verificación de 0.772 ± 0.082 en PyTorch y 0.765 ± 0.118 en Keras en 10 repartos, frente a 0.62–0.64 de las referencias sin aprendizaje.
2. **La minería de tripletas importa más que cualquier otro hiperparámetro:** con batch-hard la red colapsó (dispersión de 0.006) y con batch-all aprendió.
3. **One-shot y few-shot funcionan, con límites:** una sola foto de referencia permite identificar a la persona el 70 % de las veces (76 % en Keras), pero añadir más fotos de referencia no mejoró el resultado.
4. **Zero-shot no se logró:** con solo dos identidades de entrenamiento, la red no generaliza a una persona nueva (35.5 % de identificación frente a 33.3 % de azar).
5. **Los resultados son frágiles por el tamaño del dataset:** 38 fotos de 3 personas, con 13 de prueba, hacen que la exactitud de identificación varíe entre 54 % y 89 % según el reparto. Por eso se reportan promedios y desviaciones y no un solo número.
6. **La diferencia entre PyTorch y Keras no es concluyente** ($p = 0.807$ en verificación y $p = 0.220$ en identificación) y no debe atribuirse al framework.

Limitaciones

- El dataset es pequeño, desbalanceado (17, 7 y 14 rostros) y con fotos de muy baja resolución; la red se entrenó desde cero.
- Las fotos de una misma persona abarcan hasta diez años, lo que agrega variación de edad que la red debe aprender a ignorar.
- No se verificó la identidad de cada foto más allá de la carpeta asignada y de la revisión visual de los recortes.

Cómo mejorarlo

Más fotos por persona y más personas, y sobre todo partir de una red **preentrenada** en reconocimiento facial (por ejemplo FaceNet o ArcFace) y ajustarla con las fotos propias en lugar de entrenar desde cero; con ese enfoque el zero-shot sí es realista. Esa opción no se ejecutó aquí porque el entorno de trabajo no permitía descargar pesos preentrenados.

11. Repositorio en GitHub

Enlace: https://github.com/RubyDayana/redes-neuronales-v

Tabla 11. Contenido del repositorio

Archivo	Descripción
01 DatasetFacialPreprocesamiento.ipynb	Notebook 1, con sus salidas.
02 SiamesaPyTorchTripletLoss.ipynb	Notebook 2, con sus salidas.

Archivo	Descripción
03 SiamesaKerasTripletLoss.ipynb	Notebook 3, con sus salidas.
datos/rostros/	Rostros alineados de 112 × 112, una carpeta por persona.
datos/metadatos.csv	Confianza, tamaño, inclinación, nitidez y brillo de cada rostro.
modelos/	Pesos de la red entrenada en PyTorch (.pt) y en Keras (.keras).
resultados/	Resultados de los 10 repartos de PyTorch y de Keras (.csv).
README.md	Descripción del proyecto.

Los notebooks se ejecutan en Google Colab (PyTorch y Keras vienen instalados; MTCNN se instala con `pip install facenet-pytorch --no-deps`). Los notebooks 2 y 3 leen los rostros de `datos/rostros/`. Las fotos originales no se publican por privacidad.

12. Referencias

Hermans, A., Beyer, L., y Leibe, B. (2017). *In defense of the triplet loss for person re-identification*. arXiv:1703.07737.

Koch, G., Zemel, R., y Salakhutdinov, R. (2015). Siamese neural networks for one-shot image recognition. *ICML Deep Learning Workshop*.

Schroff, F., Kalenichenko, D., y Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 815–823.

Zhang, K., Zhang, Z., Li, Z., y Qiao, Y. (2016). Joint face detection and alignment using multitask cascaded convolutional networks. *IEEE Signal Processing Letters, 23*(10), 1499–1503.

Documentación de PyTorch (pytorch.org), TensorFlow y Keras (keras.io) y scikit-learn (scikit-learn.org).